SQL Server Transactional Replication System for Retail Database

Overview

This project implements a SQL Server transactional replication architecture to enable real-time data synchronization between OLTP and reporting systems. It separates transactional workloads from analytics, improving performance, scalability, and reporting efficiency.

Problem Statement

Traditional single-database systems face performance issues when heavy reporting queries run alongside transactional workloads. This project addresses that by separating OLTP and reporting environments.

Solution

Implemented SQL Server Transactional Replication using Publisher–Distributor–Subscriber model to ensure near real-time data movement from operational database to reporting database.

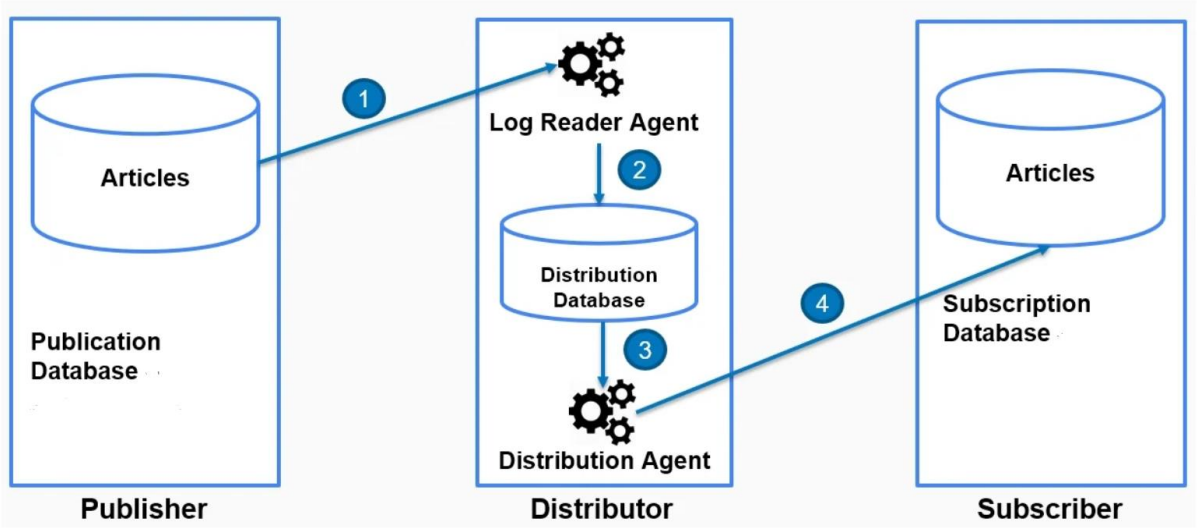
System Architecture

The solution is built on a SQL Server Transactional Replication architecture following a Publisher–Distributor–Subscriber model.

- **RetailCoreDB (Publisher):** Primary OLTP database where all transactional operations occur.
- **RetailDistDB (Distributor):** Central replication database responsible for storing replication metadata, transaction history, and agent coordination.
- **RetailReportingDB (Subscriber):** Read-only reporting database used for analytics, dashboards, and reporting workloads.

Data Flow:

1. RetailCoreDB → 2. Transaction Log → 3. RetailDistDB → 4. RetailReportingDB



In this architecture, every committed transaction in the publisher database is captured from the SQL Server transaction log. These changes are then forwarded to the distributor and subsequently delivered to the subscriber database. This ensures near real-time synchronization between operational and reporting systems while maintaining system separation for performance optimization.

Database Design Overview

The system represents a complete retail enterprise data model built using normalized relational design principles.

Core business entities include:

- Customers
- Orders and OrderItems
- Products and Suppliers
- Employees
- Warehouses and Branches

Operational and transactional entities support business continuity and traceability:

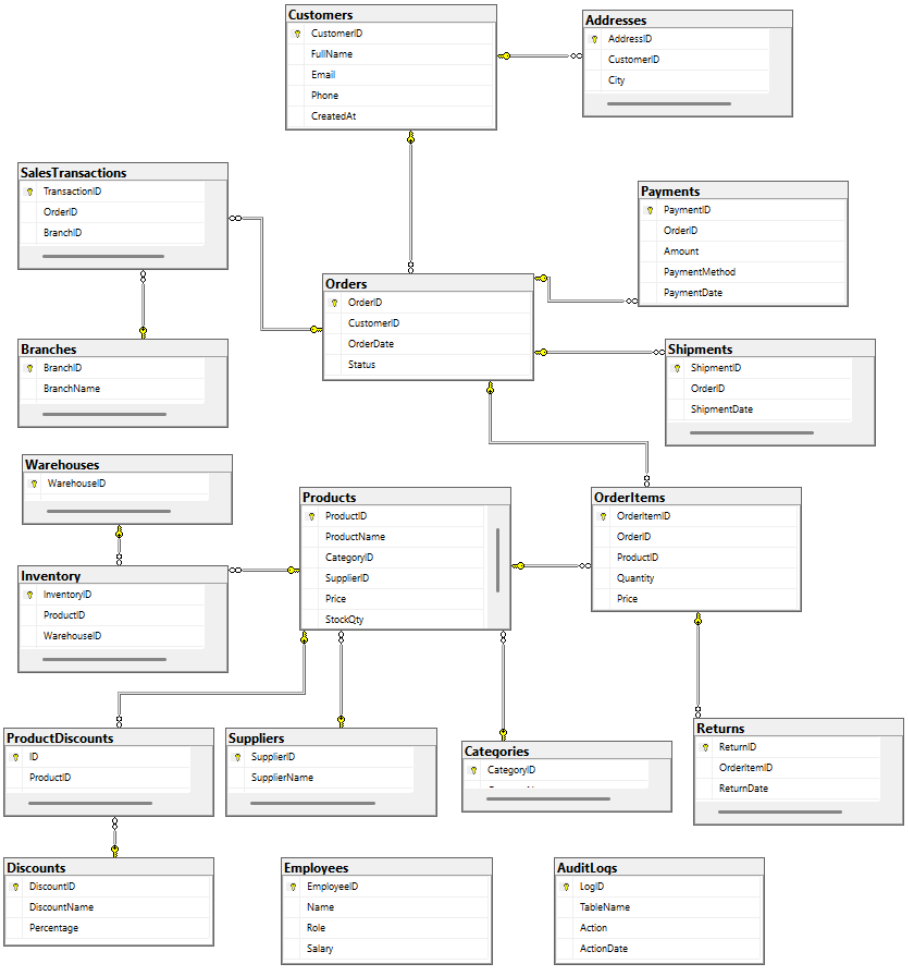
- Payments

- Shipments
- Returns
- SalesTransactions

Governance and analytical structures include:

- AuditLogs for change tracking and monitoring
- ProductDiscounts and Discounts for pricing control and promotional analysis

This structure ensures full lifecycle coverage of retail operations, from order creation to fulfillment and financial settlement.



Pre-Requisites For Implementing Replication

- Replication feature must be installed
- Entities which are part of replication must have primary keys
- No DML or DDL operations on replicated entities on subscriber

Replication Implementation

Transactional replication is configured using native SQL Server replication stored procedures and system components.

The configuration process includes:

- Initialization of the Distributor using system procedures
- Creation of the distribution database (RetailDistDB)
- Enabling the Publisher on RetailCoreDB
- Creation of a transactional publication named Publication_For_RetailCoreDB
- Registration of all core tables as replication articles using log-based capture

The system operates in continuous replication mode, ensuring that all committed transactions are propagated to the subscriber with minimal latency.

System Components

Publisher (RetailCoreDB):
Handles all operational transactions including order processing, inventory updates, and customer management.

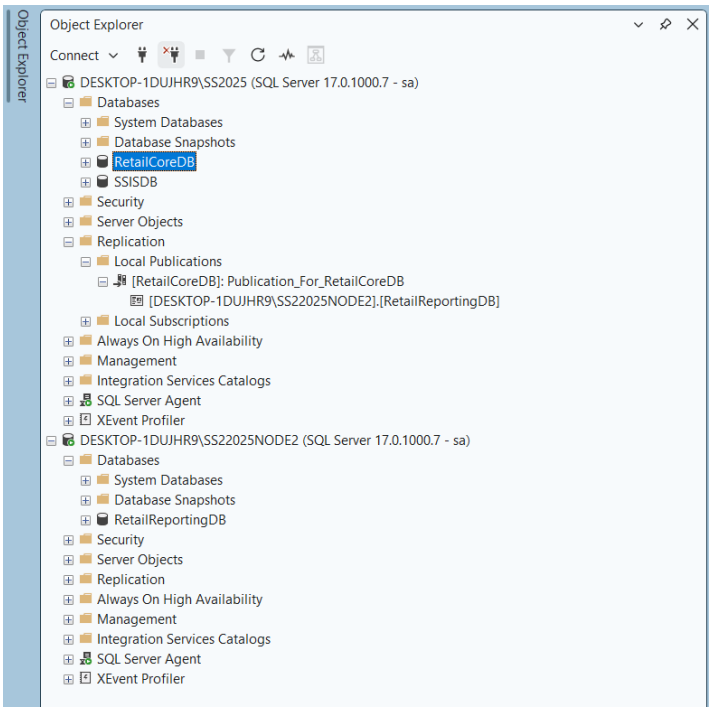
Distributor (RetailDistDB):
Acts as the replication control hub, managing replication agents, metadata, and delivery queues. Distributor can be configured by script given in the figure below.

```
/****** Scripting replication configuration. Script Date: 4/17/2026 2:44:05 PM ******/
/****** Please Note: For security reasons, all password parameters were scripted with either NULL or an empty string. ******/

/****** Installing the server as a Distributor. Script Date: 4/17/2026 2:44:05 PM ******/
use master
exec sp_adddistributor @distributor = N'DESKTOP-1DUJHR9\SS2025', @password = N''
GO
exec sp_adddistributiondb @database = N'RetailDistDB', @data_folder = N'C:\Prog[redacted], @log_folder = N'C:\[redacted]
, @log_file_size = 2, @min_distretention = 0, @max_distretention = 72, @history_retention = 48, @deletebatchsize_xact = 5000, @deletebatchsize_cmd = 2000, @security_mode = 1
GO

use [RetailDistDB]
if (not exists (select * from sysobjects where name = 'UIProperties' and type = 'U '))
    create table UIProperties(id int)
if (exists (select * from ::fn_listextendedproperty('SnapshotFolder', 'user', 'dbo', 'table', 'UIProperties', null, null)))
    EXEC sp_updateextendedproperty N'SnapshotFolder', N'C:\Program Files\Microsoft SQL Server\MSSQL17.SS2025\MSSQL\ReplData', 'user', dbo, 'table', 'UIProperties'
else
    EXEC sp_addextendedproperty N'SnapshotFolder', N'C:\Program Files\Microsoft SQL Server\MSSQL17.SS2025\MSSQL\ReplData', 'user', dbo, 'table', 'UIProperties'
GO

exec sp_adddistpublisher @publisher = N'DESKTOP-1DUJHR9\SS2025', @distribution_db = N'RetailDistDB', @security_mode = 0, @login = [redacted], @password = [redacted]
@working_directory = N'C:\Program Files\Microsoft SQL Server\MSSQL17.SS2025\MSSQL\ReplData', @trusted = N'false', @thirdparty_flag = 0, @publisher_type = N'MSSQLSERVER'
GO
```

Subscriber (RetailReportingDB):

Functions as a reporting layer optimized for read-only analytical queries and business intelligence workloads.

```
----------------BEGIN: Script to be run at Publisher 'DESKTOP-1DUJHR9\SS2025'----------------
use [RetailCoreDB]
exec sp_addsubscription @publication = N'Publication_For_RetailCoreDB', @subscriber = N'DESKTOP-1DUJHR9\SS22025NODE2', @destination_db = N'RetailReportingDB',
@subscription_type = N'Push', @sync_type = N'automatic', @article = N'all', @update_mode = N'read only', @subscriber_type = 0
exec sp_addpushsubscription_agent @publication = N'Publication_For_RetailCoreDB', @subscriber = N'DESKTOP-1DUJHR9\SS22025NODE2', @subscriber_db = N'RetailReportingDB',
@job_login = null, @job_password = null, @subscriber_security_mode = 0, @subscriber_login = [redacted] @subscriber_password = [redacted] @frequency_type = 64, @frequency_interval = 0,
@frequency_relative_interval = 0, @frequency_recurrence_factor = 0, @frequency_subday = 0, @frequency_subday_interval = 0, @active_start_time_of_day = 0,
@active_end_time_of_day = 235959, @active_start_date = 20260417, @active_end_date = 99991231, @enabled_for_syncmgr = N'False', @dts_package_location = N'Distributor'
GO
----------------END: Script to be run at Publisher 'DESKTOP-1DUJHR9\SS2025'----------------
```

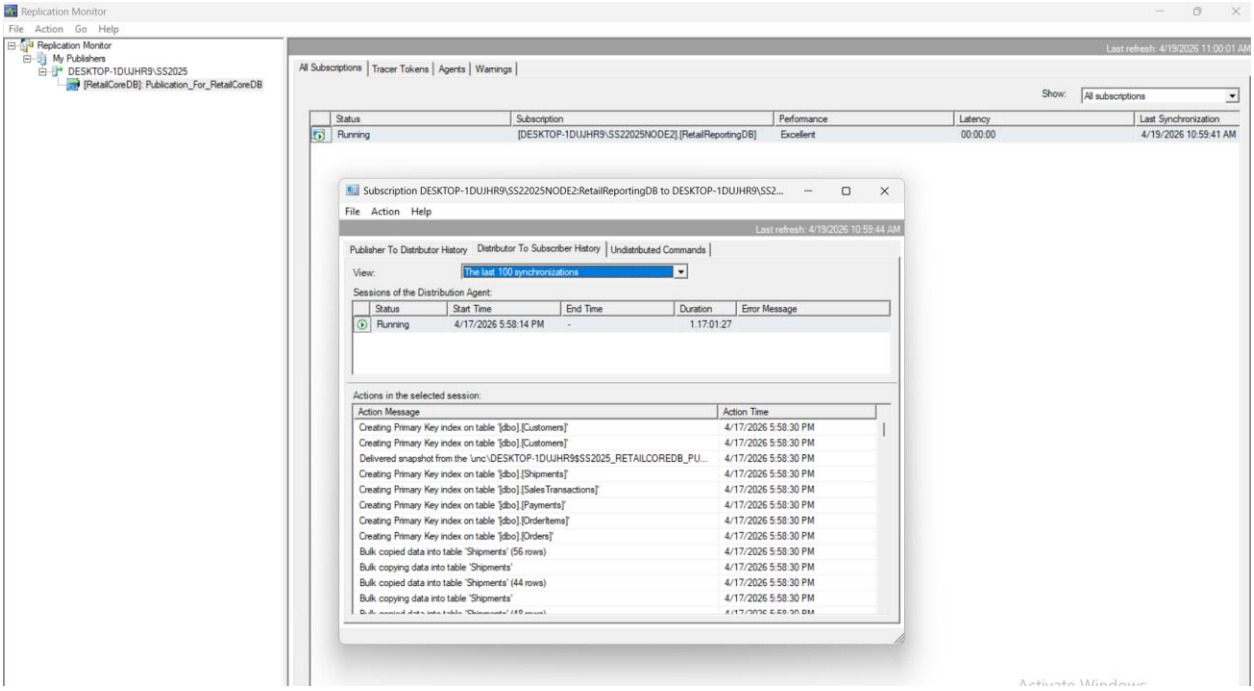


Data Flow Process

The replication workflow follows a structured pipeline:

1. Transactions are executed in the RetailCoreDB database.
2. The Log Reader Agent captures committed changes from the transaction log.
3. Changes are transferred to the distribution database (RetailDistDB).
4. The Distribution Agent applies these changes to the subscriber database.
5. RetailReportingDB is updated in near real time for reporting and analytics.

This ensures consistency between operational and reporting systems without impacting OLTP performance.



Security and Reliability

The replication environment is secured using SQL Server authentication for replication agents. The subscriber database is configured with read-only permissions to prevent unauthorized modifications.

Additionally, snapshot folders and replication working directories are restricted at the operating system level to prevent unauthorized access or tampering.

This design ensures both data integrity and controlled replication execution.

Implementation Summary

The system is implemented using SQL Server native replication procedures, including:

- Distributor configuration procedures
- Distribution database setup
- Publication creation for RetailCoreDB
- Article definition for all replicated tables

- Push subscription configuration for RetailReportingDB

This setup enables automated, continuous, and reliable data synchronization between OLTP and reporting layers.

Replicated Entities

The following entities are included in the replication scope:

Customers, Employees, Products, Suppliers, Categories, Orders, OrderItems, Payments, Shipments, Returns, Inventory, Warehouses, Branches, SalesTransactions, Discounts, ProductDiscounts, AuditLogs, and Addresses.

These tables collectively represent the full operational and analytical dataset of the retail system.

Key Features

- Real-time data synchronization
- Separation of OLTP and reporting workloads
- Push subscription configuration
- Continuous replication
- DDL replication enabled

Technologies Used

- Microsoft SQL Server
- T-SQL
- SQL Server Agent
- Transactional Replication

Challenges & Solutions

Faced foreign key constraint issues during data load; resolved by inserting master tables first and maintaining referential integrity.

Outcome

This implementation successfully demonstrates an enterprise-grade data replication system using SQL Server Transactional Replication. Achieved near real-time reporting without impacting OLTP performance. Demonstrated scalable enterprise data architecture.

Note: All the configuration sql scripts are attached in the sql file.